

硬核拆解 OpenClaw：如何构建真正稳健的生产级 Agent 系统？【下】

mp.weixin.qq.com/s/6H3PW5a-P1965lLg5PIhxA

秋山墨客

Original

拒绝 Toy，构建 System

专注企业级 AI 应用架构与深度工程实践

点击上方，加入我们 ↗

本文延续上篇（[硬核拆解 OpenClaw：如何构建真正稳健的生产级 Agent 系统？【上】](#)），继续拆解 OpenClaw 中 Agent 模块的一些工程方法，以帮助我们构建真正的生产级 Agent 系统。

安全与风险防御

记忆与状态管理

成本与效能优化

子 Agent 协作机制



01

安全与风险防御

作为一个高权限的桌面 Agent，能力越强，风险越大。大模型本身具有不可预测性，再叠加提示注入等攻击可能，如果没有系统性的风险防御，Agent 的“手脚”（工具）越多，越可能把一次普通的请求变成事故。



管住“手脚”：零信任工具策略

对 OpenClaw 来说，最大的风险来自工具本身。在这类 Agent 中，文件读写、Shell 脚本、代码执行往往是标配，但也同时是主要风险源。比如你把 OpenClaw 开放给某个工作群，有人发来请求：

“帮我测试这个 Python 脚本（含有某个高危动作）”

如果 Agent 直接照做，后果可能是灾难性的。

【OpenClaw 的方法】

OpenClaw 通过“Tool-policy”对工具使用做权限管控。其关键点在于：权限控制不是在调用工具那一刻才做，而是在构建 Agent 上下文时就完成了——

当消息到达 Gateway、准备构建 Agent 上下文时，系统会基于 ChannelId / UserId / GroupId 等信息，动态生成本次会话可用工具的“白名单”列表，再把这个白名单交给 Agent。管控的大致过程如下。

首先是策略配置（示例）：

```
channels:
  telegram:
    groups:
      "-100123456": # 公开群 (Public)
        tools:
          profile: "minimal" # 默认只读，禁止 Shell/File Write
          allow: ["web_search"]
          toolsBySender: # 针对特定用户的特权覆盖
            "admin_user":
              profile: "full" # 管理员拥有全权限
      "-100987654": # 内部开发群
        tools:
          profile: "coding" # 允许代码执行
```

接着在调用 Agent 前，会做策略解析与工具匹配：

1. 先检查当前群组配置中的 toolsBySender 是否命中发送者
2. 若未命中，使用群组的 tools 配置
3. 若群组也未配置，回退到全局默认策略

最后给 Agent 注入可用工具。一旦 Agent 收到的工具集里不包含某些能力：它们不会出现在 Agent 上下文中（通常在System Prompt 的工具部分）

对应的工具实现函数的句柄也会被从 Agent 中移除

这使得很多提示注入技巧即便“骗”过了模型，也没有办法从系统层面绕过。

【关键解读】

Agent 安全控制必须从提示词约束上升到工程约束。而 Tool-policy 本质上是一套基于身份与场景的访问控制列表（ACL）。放到企业级 Agent 的生产环境里，它的参考意义在于：

Agent 默认都不可信：最有效的防风险方式，是不给它“武器”（工具）

能用哪些工具，不由 Agent 决定：而由授权中心（Gateway）决定

授权必须随场景实时生效：同一用户切换环境（比如到不同群组/任务），也必须重新计算权限

当然，安全管控要求企业具备可运营的“工具分级 + 策略配置 + 审计机制”来配合，否则也容易失控（过松导致事故，过紧影响可用性）。

管住“临时工”：子 Agent 的能力限制

子 Agent（Subagent）是很多平台/框架的常见机制（如 Claude、LangChain 等）。在 OpenClaw 中，主 Agent 可以通过 sessions_spawn 工具派生 Subagent 执行长时的复杂任务。但如果 Subagent 继承主 Agent 的全部能力，就会带来安全隐患：

递归“炸弹”：Subagent 再次调用 sessions_spawn，导致无穷嵌套

越权操作：一些高权限任务应由主 Agent 统一协调与完成

【OpenClaw 的方法】

OpenClaw 对 Subagent 采用“默认收回敏感与高危工具”的策略；对特别危险的工具，再叠加二次检查，确保无法绕过。

代码层面可以看到默认 deny 列表（示例）：

```
const DEFAULT_SUBAGENT_TOOL_DENY = [  
  // 会话管理 - 主Agent统一协调  
  "sessions_list",  
  "sessions_history",  
  "sessions_send",  
  "sessions_spawn", // 阻止递归生成子Agent // 系统管理  
  "gateway",  
  "agents_list",  
  "whatsapp_login", // 调度与状态  
  "session_status",  
  "cron", // 记忆  
  "memory_search",  
  "memory_get",  
];
```

这些工具会从子代理的工具列表中彻底剔除，子代理的 System Prompt 和运行时工具函数中都看不到它们。

另外，对于如 sessions_spawn 这样的危险工具，会在 Agent 层做再次拦截：

```
if (isSubagentSessionKey(requesterSessionKey)) {
  return jsonResult({
    status: "forbidden",
    error: "sessions_spawn is not allowed from sub-agent sessions",
  });
}
```

【关键解读】

最小权限原则不仅适用于人类用户，也适用于 AI。Subagent 更像工作中的“临时工”：它的职责是把某个子任务做完。因此需要给 Subagent 明确边界：它负责执行，但很多管理权不下放，由主 Agent 统一收口与协调。

当然也要把握好副作用：如果 Subagent 的工具过少、自主性过低，一些子任务可能无法独立闭环，导致频繁回到主 Agent 拆解与重试，反而拖慢整体效率。解决思路通常不是简单“放权”，而是把子任务定义得更清晰（输入/输出边界明确、允许的操作明确、失败时的回退路径明确），让“临时工”在有限工具下也能稳定交付。

02

记忆与状态管理

长期记忆是 OpenClaw 的核心能力之一。虽然市面上已有不少开源记忆框架（常见是向量存储 / 图存储），但真正集成到可长期运行的 Agent 里，细节仍然很多：记忆分哪些类型、什么时候写入、如何提取/压缩、什么时候加载、如何避免“该记的没记、该忘的记住了”等。

【OpenClaw 的方法】

OpenClaw 自己实现了一套 Markdown 文件 + SQLite 向量索引 的记忆机制，大致可以概括为：

1) 两类记忆分工

每日工作记忆：用命名为 YYYY-MM-DD.md 的文件保存，更像“工作日志”，用于实时回顾会话过程，带时间线与上下文，允许重复。

MEMORY.md：保存提炼后的“精华”长期记忆，比如从工作记忆中去重、归类、抽取出的关键偏好与结论，长期有效。

两者的定位可以理解为：

一个记录“发生了什么”，一个记录“我应该长期记住点什么”。

2) 记忆混合检索

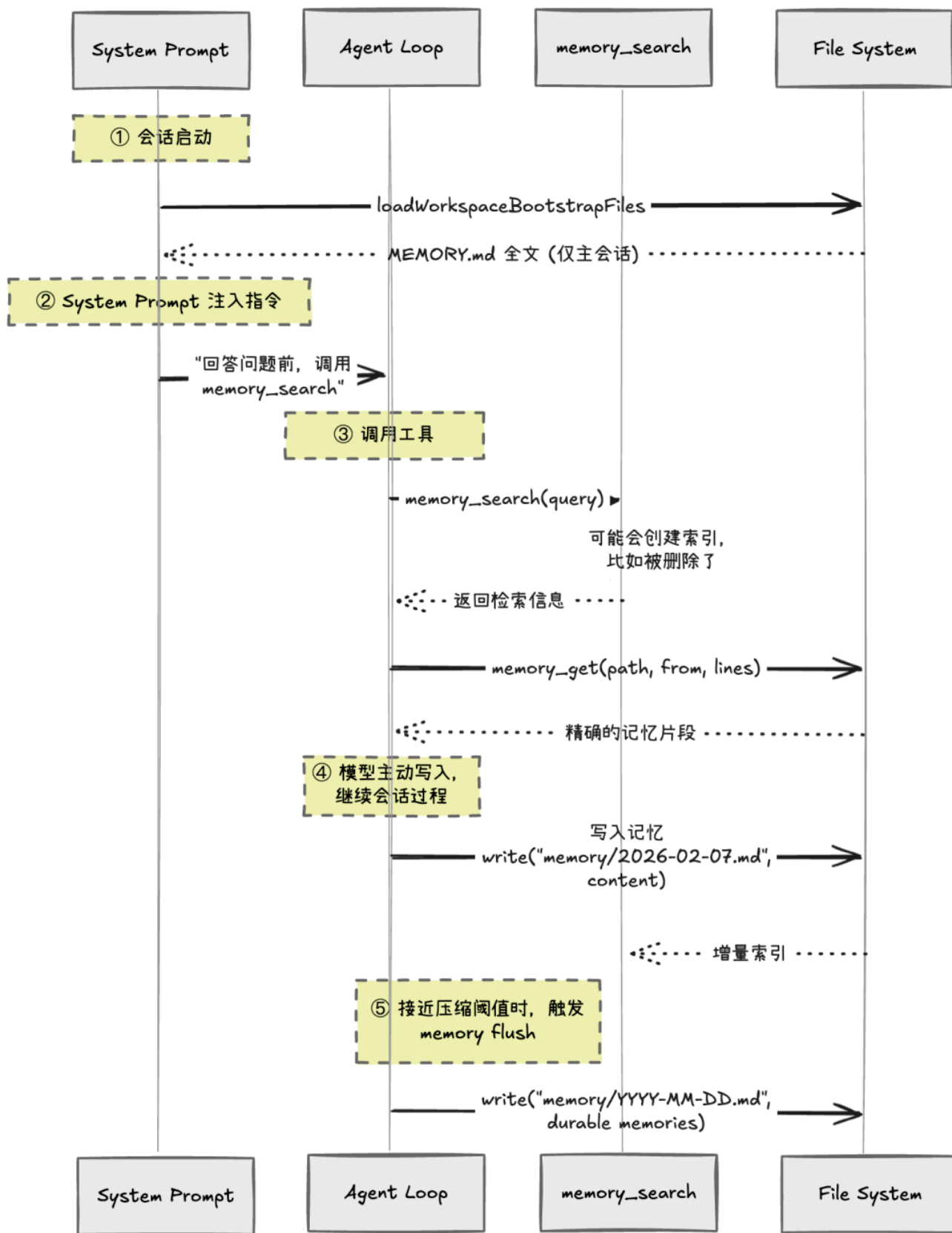
整体思路接近 RAG 的过程：创建向量索引（SQLite + vector 扩展）→ 向量检索 + BM25 混合检索 → 加权评分排序 → 读取记忆上下文。

在每次检索记忆时，也可能触发索引的创建/更新（例如索引文件被误删时自动恢复）。

3) 压缩前的 Memory Flush

由于 OpenClaw 有上下文守卫机制：当上下文逼近阈值时，可能触发会话压缩（摘要），从而丢失部分对话细节。为降低这个风险，系统会在压缩之前先执行一次 Memory Flush — 相当于“给 Agent 一次把关键信息写入记忆的机会”。

整个时序如下：



【关键解读】

相比直接集成某个现成的记忆框架，OpenClaw 这套机制的关键特点是：用可读的 Markdown 做“记忆本体”，向量索引用来加速检索。

好处：记忆随时可读、可编辑，便于解释与追溯 Agent 的行为，必要时还能人工修正“记错的东西”。

不足：文本记忆带来可能的隐私与合规风险（比如工作日志类记忆可能包含敏感信息）。结合 OpenClaw 的“个人桌面工具”为主的定位，这种取舍可以理解，但在企业场景里需要考虑更严格的读取权限、脱敏与留存策略。

03

成本与效能优化

很多推理模型（如 Claude、Gemini 等）的“思考”过程既贵又慢。对“你好”“查个天气”这类简单请求，如果也触发深度思考，就是明显浪费；但面对复杂编程或长程任务，如果不让模型认真推理，直接回答的“翻车”概率又会明显升高。所以，你需要有一个成本与效能平衡的机制。

【OpenClaw 的方法】

OpenClaw 采用了可选思考分级机制：在系统中定义了 6 个递增的思考等级，用来对应不同复杂度的任务：

等级	适用场景举例
off	"你好"、查时间、简单翻译
minimal	快问快答、格式转换、简单总结
low	默认。日常对话、写邮件、代码补全
medium	代码 Debug、多步分析、方案对比
high	架构设计、复杂算法、长文推理
xhigh	仅限 gpt-5.2-codex 等特定模型

需要注意的是，不同模型对“思考等级”的支持差异很大：有的支持多档位的思考级别、有的只有思考的开/关、有的则完全不支持。因此 OpenClaw 会根据所选模型，把内部等级映射到对应的模型 API 参数。

当一条消息（任务）进入系统后，思考等级会按一定优先级确定：

优先级	来源	示例	生效范围
①内联指令	消息中带 <code>/think high</code>	帮我优化这段代码 <code>/think high</code>	本条消息
②Session指定	单独发送 <code>/think medium</code>	发送纯 <code>/think medium</code> （无内容）	后续消息
③Agent配置	<code>agents.defaults.thinkingDefault</code> 配置	<code>openclaw.json</code> 中配置	全局默认
④模型兜底	检查模型信息目录来决定等级	如果模型支持 Reasonning，否则 → <code>off</code>	全局配置

另外，如果运行时遇到 API 返回类似“当前模型不支持该思考等级”的错误，系统会解析错误信息并自动降档重试，尽量避免任务直接失败。

【关键解读】

很显然：不是所有问题都值得让模型深度思考 — 简单查询和复杂方案设计的推理成本，可能相差 N 倍。OpenClaw 的做法是：把控制权交给用户，让用户按任务风险与成本权衡，灵活调整思考级别，并用自动降档兜底提升可用性。

但它也有代价与不足：对外部用户（尤其是客户）来说，让用户自己选思考等级可能增加体验负担，也不一定很自然；并且用户如果长期选择低等级，会导致任务质量不稳定。

一种可能的企业实践是在可控与体验之间找平衡 — 例如在系统侧按任务类型、风险等级或客户等级做默认策略；再把手动选项作为少数高级入口，而不是让每一次对话都由用户决策。

04

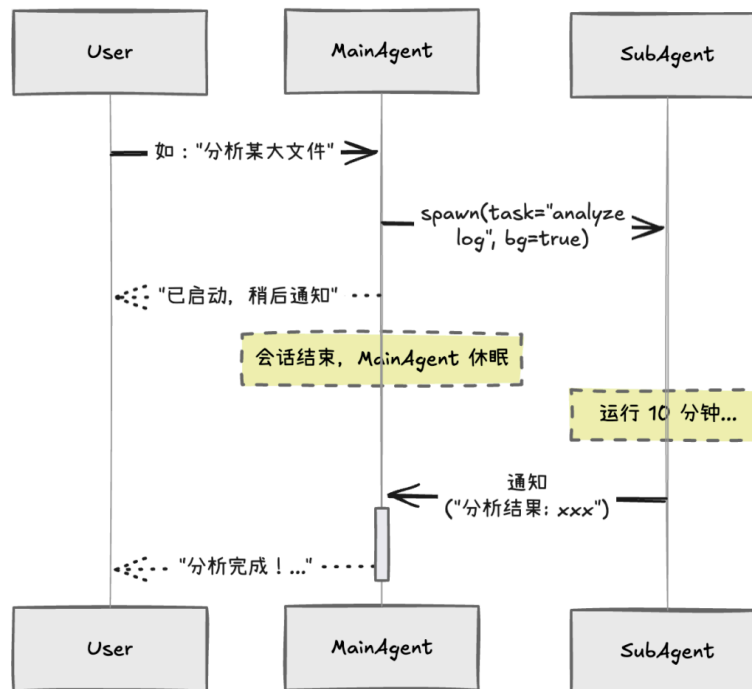
子 Agent 协作机制

最后来说说 OpenClaw 的 Subagent 机制（OpenClaw 本身的推理循环就是 ReAct，并没有太多的奥秘）。

面对复杂、长时间任务，如果让主 Agent 同步等待，很容易触发 HTTP 超时、阻塞用户交互，也会拉低整体吞吐量。这时就可以引入 Subagent 机制，把长时间的复杂任务拆出去后台跑。

【OpenClaw 的方法】

当任务较复杂时，主 Agent 可以派生一个或多个异步 Subagent 协助完成，大致流程是：



1. 派生：主 Agent 为复杂任务派生 Subagent，并设置为后台/异步模式
2. 通知：主 Agent 先回复用户“任务已启动...”，然后结束当前会话
3. 回调：Subagent 任务完成后回传结果，由主 Agent 汇总并推送给用户

【关键解读】

Subagent 机制在 Claude、LangChain 等框架中都很常见，本质上是一种“临时且受限”的多智能体协作：主要收益是 不阻塞主 Agent、可并行加速、上下文隔离，对工程可用性非常关键。

不过，OpenClaw 目前更多依赖 LLM 的提示来决定是否派生 Subagent：

If a task is more complex or takes longer, spawn a sub-agent...

这意味着可能出现误判：该派生时没派生导致超时，不该派生时派生又引入额外开销与复杂度。

要在企业级场景里更稳妥一些，可以考虑做一些增强：

给出更具体的决策框架：补充更可执行的判断信号，例如：预估步骤数、是否包含多轮外部调用、是否需要大规模检索/爬取等。

让工具/技能给出一些“成本提示”：比如在工具信息里增加预估耗时、复杂度等提示，并告诉模型：当任务可能涉及“慢工具”组合时则派生 Subagent。

用历史数据做校准：记录每次任务的输入、执行时长、成本等；新任务到来时，参考相似的历史任务来校准判断，而不是只靠模型当时的直觉。

另外也要注意 Subagent 的代价：它会引入更多的会话管理、结果汇总、失败回退与并发控制问题（尤其是多个 Subagent 并行时）。因此实践上通常要配套一些超时、重试策略、以及主 Agent 的管控责任，让这套机制运行更稳定。

至此，我们对 OpenClaw 的一些工程实践进行了拆解与分析。

其中不少实践值得借鉴，但由于 OpenClaw 更偏个人工具定位，落到企业场景仍需结合业务需要、合规要求与运维成本等做取舍与改造。

归根结底，生产级 Agent 和 Demo 的差别，不在“能不能跑起来”，而在于工程上能否做到**安全可控、可靠可用、可解释**；一旦出问题，能够**回退、追责、持续运行** — 这需要我们持续的学习、验证与改进。

END

喜欢就关注哦



动动小手点个赞



点在看最好看



加入公众号交流群（说明来意）



秋山墨客



扫一扫上面的二维码图案，加我为朋友。

公众号 · AI大模型应用实践